

Embedded Software Engineering

Simon Künzli, Tobias Welti
HS 2025

Agenda HS25

Vorlesung (welo)

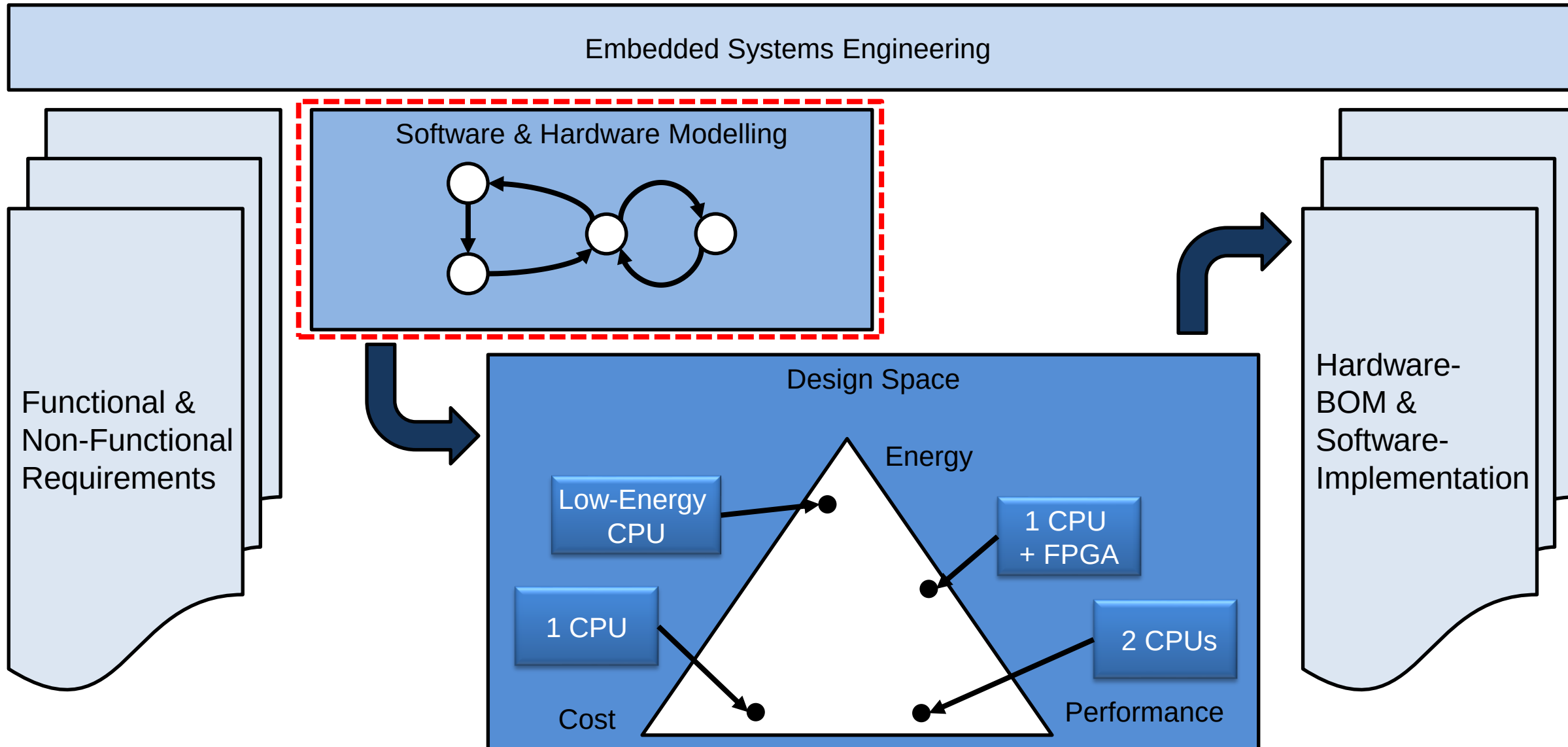
Vorlesung (kuex)

Übungen / Praktika

Stand:
27.08.25

wk #	Do, 10:00 - 11:35	Do, 08:00 – 09:35 / Do 12:00 – 13:35	
01 15.09.	Modulorganisation & Einführung Embedded Systems	No Lab	
02 22.09.	SW-Paradigmen für Embedded Systems	No Lab	P1: First steps U96
03 29.09.	Requirements für Embedded Systems	P1: First steps U96	P1: Finish Lab
04 06.10.	Software-Entwicklung für ES (Modellierung, SW-Specs)	P1: Finish Lab	U1: Requirements
05 13.10.	Entwicklungsprozesse / CI/CD	U1: Requirements	U2: Modeling
06 20.10.	Non-functional Requirements	U2: Modeling	U3: Implementation & Test
07 27.10.	Visite Fertigung Siemens	Visite Fertigung Siemens	
08 03.11.	Fokus "Energie« / andere Prozessoren?	U3: Implementation & Test	P2: Energy Consumption
09 10.11.	Fokus "Performance"	P2: Energy Consumption	P3: AES in SW, incl. Measurement on R5
10 17.11.	Einführung FPGA	P3: AES in SW, incl. Measurement on R5	P3: Finish Lab
11 24.11.	Einführung Design Space Exploration (DSE)	P3: Finish Lab	P4: AES in FPGA and HW-Accelerator
12 01.12.	DSE: Kostenfunktionen und Suche im Entwurfsraum	P4: AES in FPGA and HW-Accelerator	P4: Finish Lab
13 08.12.	Einführung RTOS	P4: Finish Lab	P5: DSE
14 15.12.	Dual-Prozessor-Systeme mit Uniform Memory Access	P5: DSE	No Lab

Advance Organizer ESE



Learning Objectives

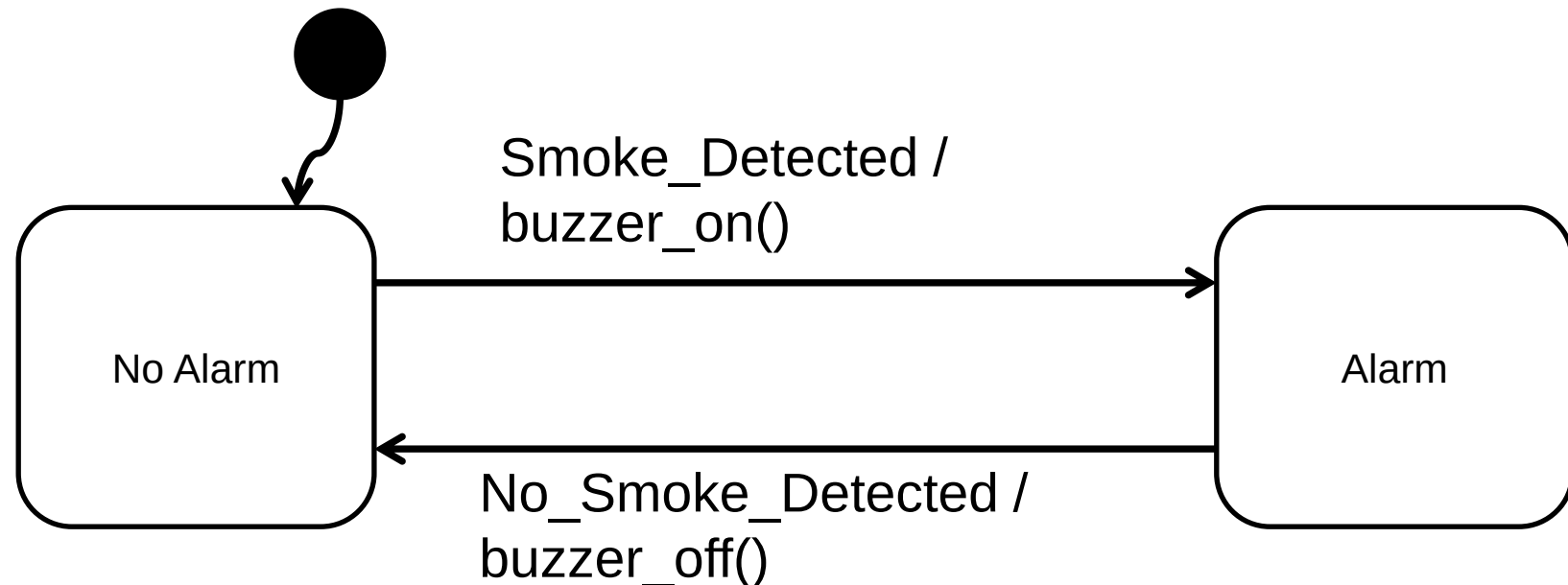
After today's lesson, ...

- ... the students know modeling techniques for Embedded Systems Hardware and Software and can explain the benefits of modeling
- ... the students can model Embedded Systems' behavior using the UML state machine description
- ... the students know simple model verification techniques and can explain their benefits
- ... the students know the possibility of generating code from SW models automatically
- ... the students know the concept of tasks and their binding to HW resources

Recap: Ingredients to UML State Machines

■ In UML State Machines, we know 4 types of elements:

- States
- Transitions
- Events
- Actions

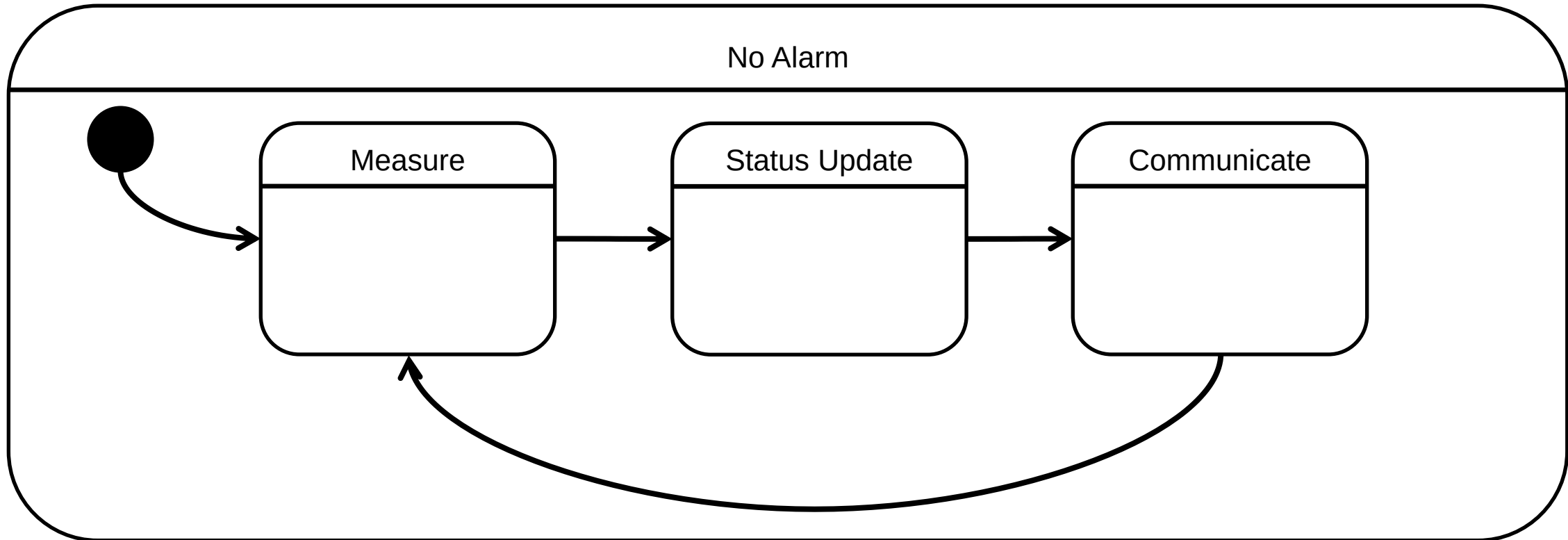


UML State Machines

- **Unified modeling language (UML) can be used to visualize a software using a collection of diagrams:**
 - Structural diagrams, such as class diagrams, components diagrams, deployment diagrams, etc.
 - Behavioral diagrams, such as sequence diagrams, use-case diagrams, communication diagram, **state machine diagrams**, etc.
- **Each of the diagram types has its specific purpose**
- **A state machine diagram describes the system's behavior in response to external stimuli (exactly what an ES does...)**

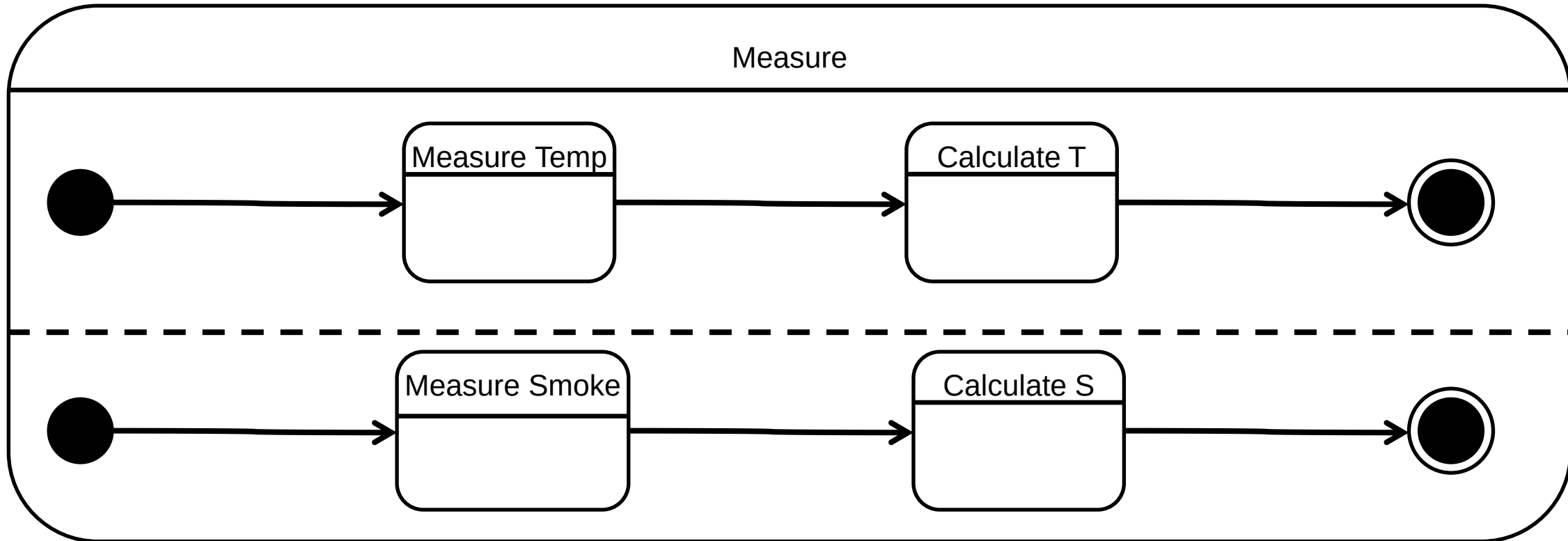
Composite States

- Composite states contain substates and contain one or more regions
- Hierarchy can be achieved with composite states that encompass other states
→ lower-level details are represented by several “internal” states in one high-level state



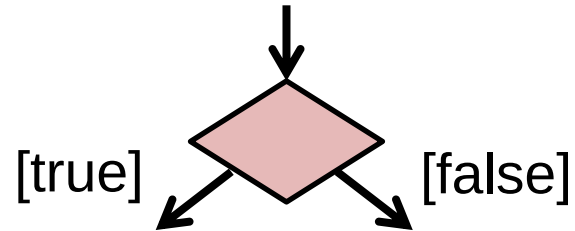
Composite States

- Orthogonal states contain more than one region
- Concurrency can be achieved with multiple regions that execute concurrently



Pseudo States

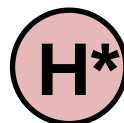
- **Choice:** Conditional branch, guards of its outgoing arches are evaluated

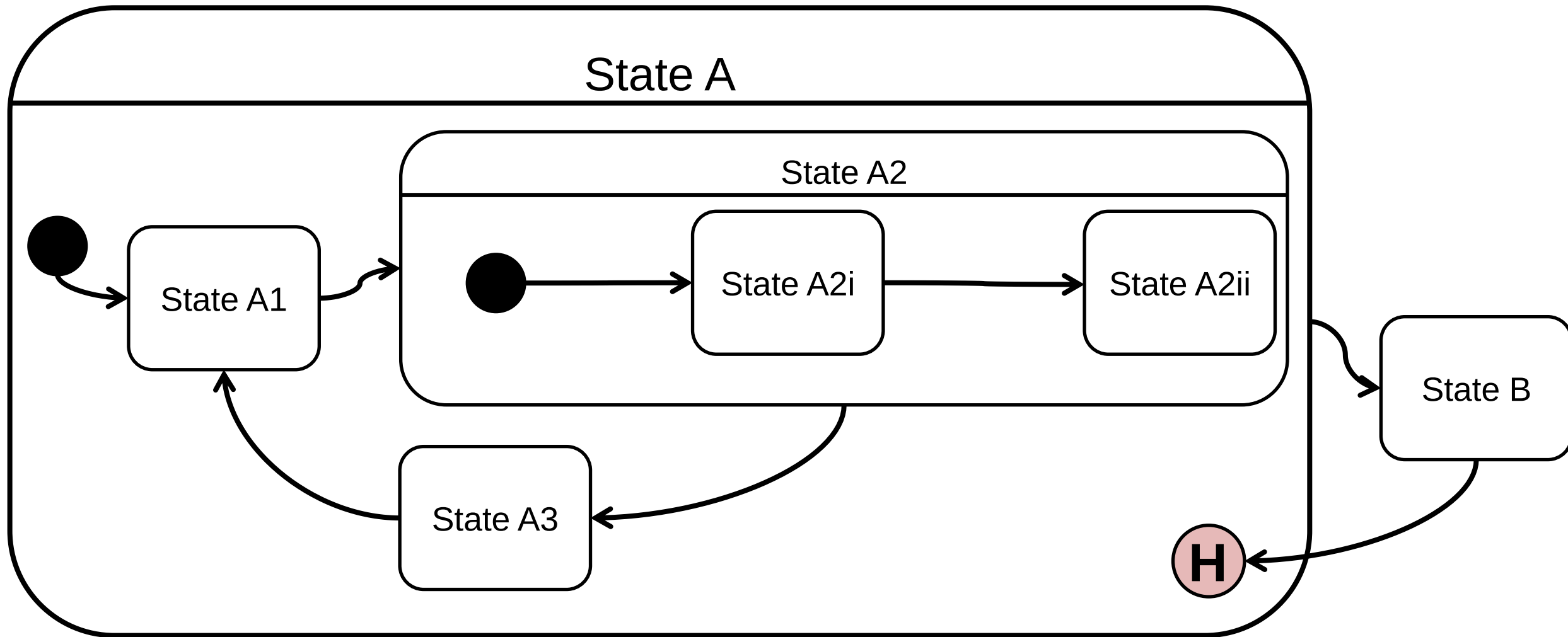


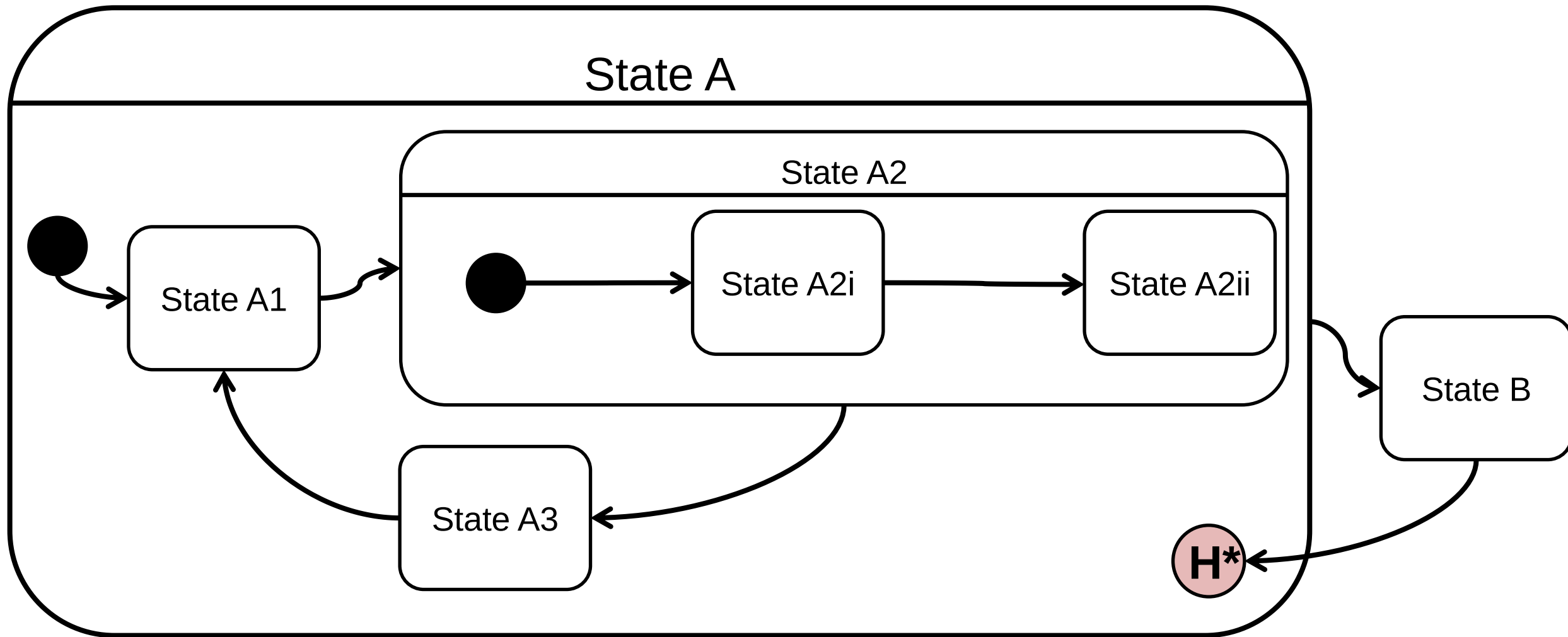
- **Shallow history:** Records history only of composite state it belongs to



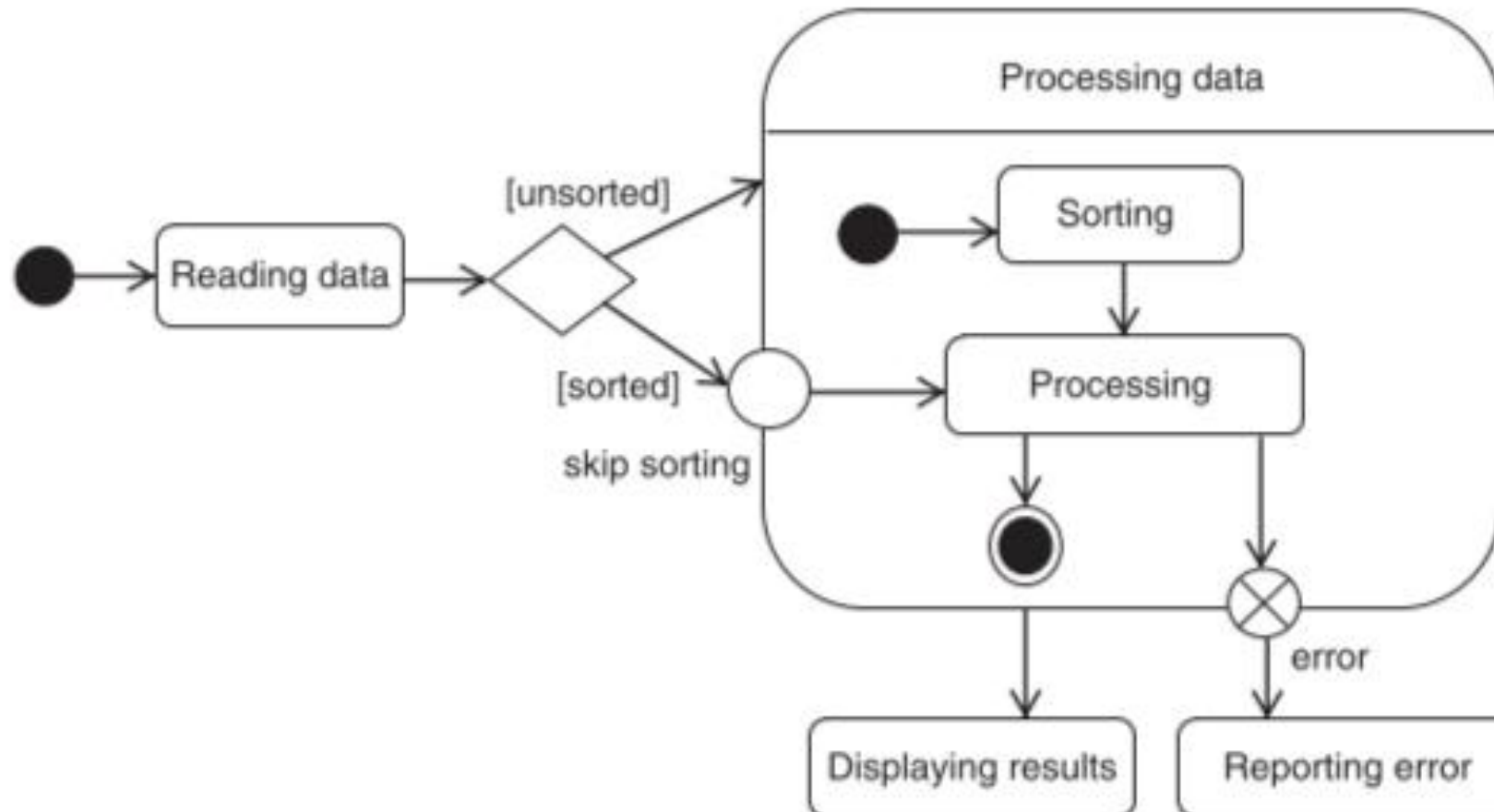
- **Deep history:** Records history of sub-states as well







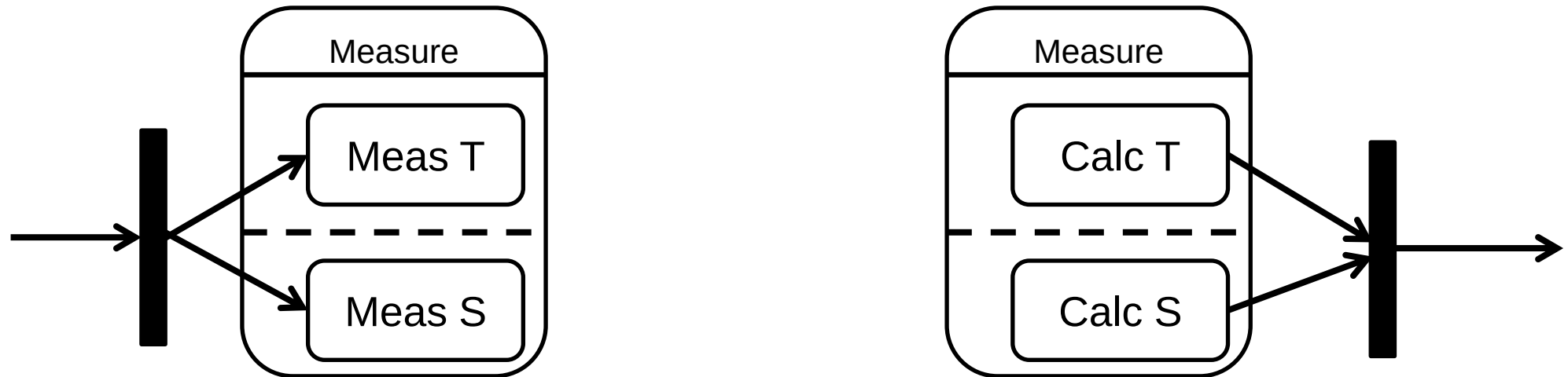
- Entry & Exit Point



Source: Jiacun Wang: Real-Time Embedded Systems

Fork/Join & Terminate

- **Fork & Join:** No guards on outgoing edges of forks, no guards on incoming edges of joins allowed. Used for transitions to specific states in orthogonal regions



- **Terminate:** Special pseudo-state to represent the complete stop of a state machine. In other words, when the *terminate* state has been reached, the execution of the system has been terminated.



Rules for UML State Machines

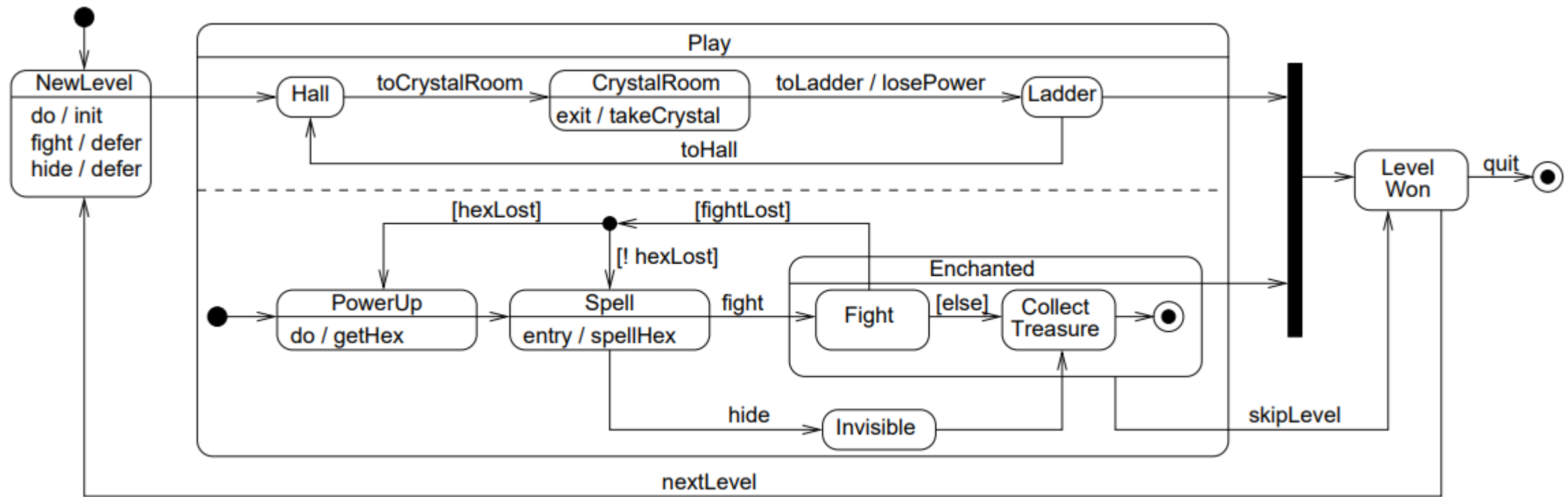
■ A UML state machine ...

- ... must be deterministic, i.e., for each state and for each trigger, the reaction must be determined
- ... must have an initial state, and so does a nested state
- ... should not have infinite completion cycles
- ... should not contain unreachable states

■ But a UML state machine ...

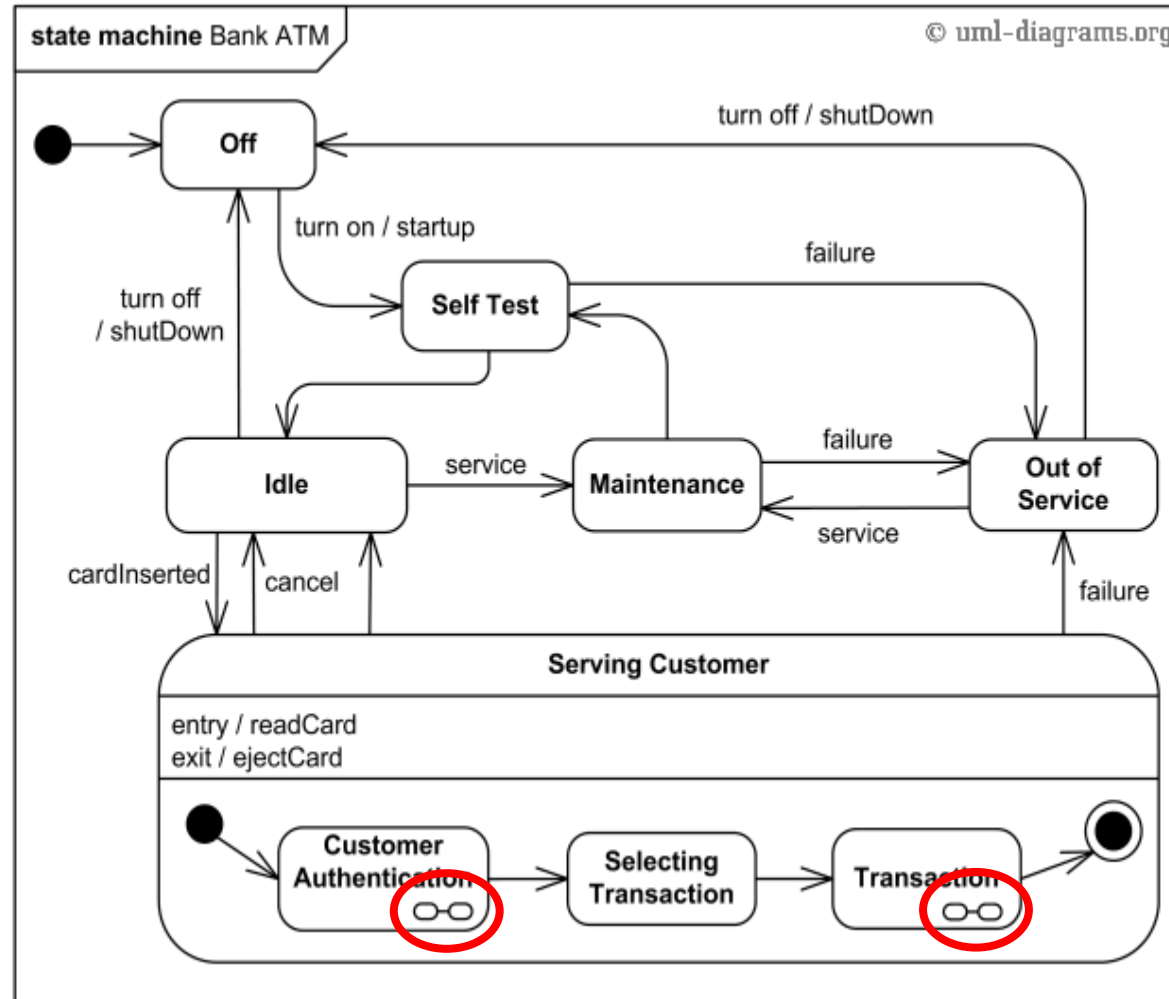
- ... may have self-transitions
- ... may have states with no out-transitions
- ... may have transitions without trigger, i.e., completion transitions

Examples for UML State Machine



(a) Movement and basic actions

Examples for UML State Machine



Modelling – Summary

- **Finite State Machines (FSM) can be used to model reactive systems and are relatively simple to implement**
- **Complexity of FSMs may grow to unwieldy, incomprehensible systems as the concepts of hierarchy and concurrency are missing**
- **UML State Machines introduce these handy concepts besides other useful extensions to enable the specification of complete SW models for ES**

➔ We will now take a deep dive into model verification and model-based code generation (based on UML State Machines)

Recap: Benefits of Modeling



13.4 Überwachung des Programmablaufs

(siehe auch Anhang J)

13.4.1 Der Ablauf des Programms muß überwacht werden. Die Überwachungseinrichtung muß eine Systemstörung signalisieren, wenn Routinen, die im Zusammenhang mit den Hauptfunktionen des Programms stehen, nicht mehr innerhalb 100 s abgearbeitet werden.

13.4.2 Die Funktion der Überwachungseinrichtung sowie die Signalisierung einer Störung darf nicht durch eine Ablaufstörung im Programm des überwachten Systems verhindert werden.

13.4.3 Bei Erkennen einer Ablaufstörung wie in 13.4.1 angegeben, muß die BMZ innerhalb 100 s einen sicheren Betriebszustand einnehmen. Dieser sichere Betriebszustand muß vom Hersteller definiert werden.

13.4.4 Die Überwachungseinrichtung muß die höchste vorgesehene Prioritätsebene verwenden, um die BMZ in den in 13.4.3 angegebenen sicheren Betriebszustand zu bringen (z. B. den höchsten non-maskable interrupt, NMI).

- What would be interesting questions to our model for the washing machine?
- Is there a way to prove a specific behavior of a system?

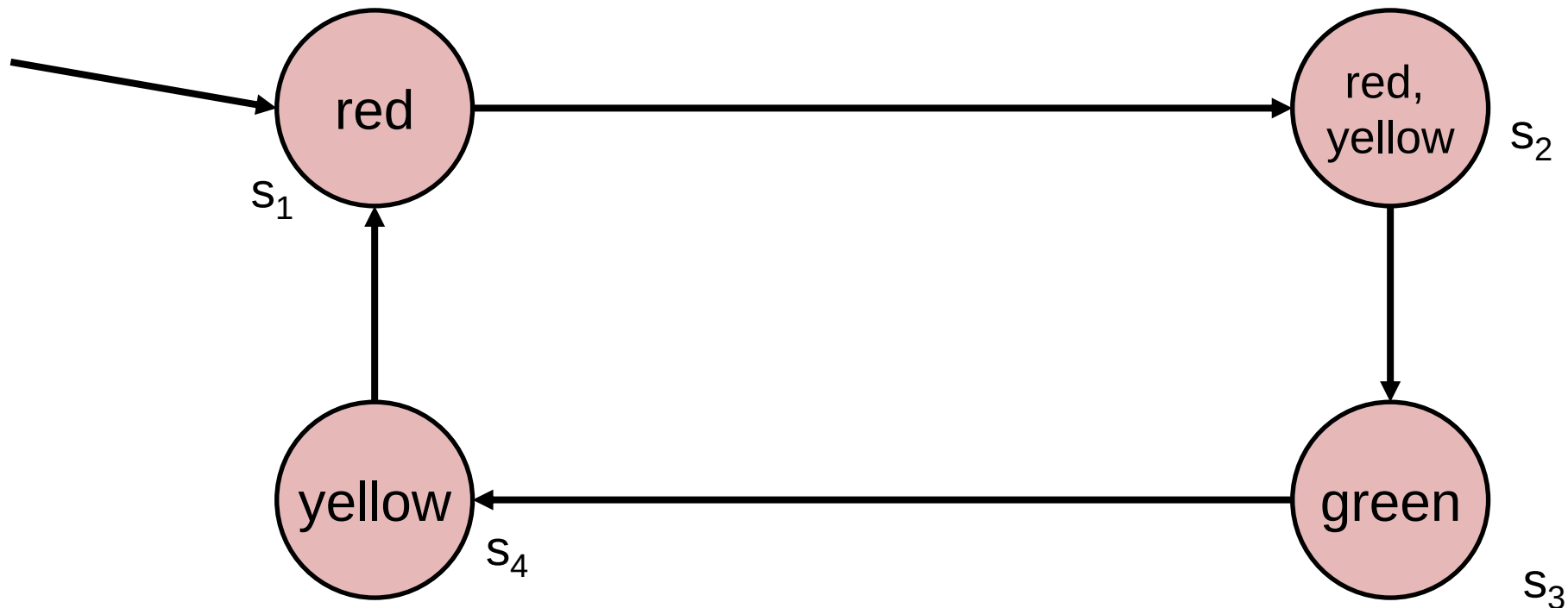


Source: Colourbox.de

State Transition System

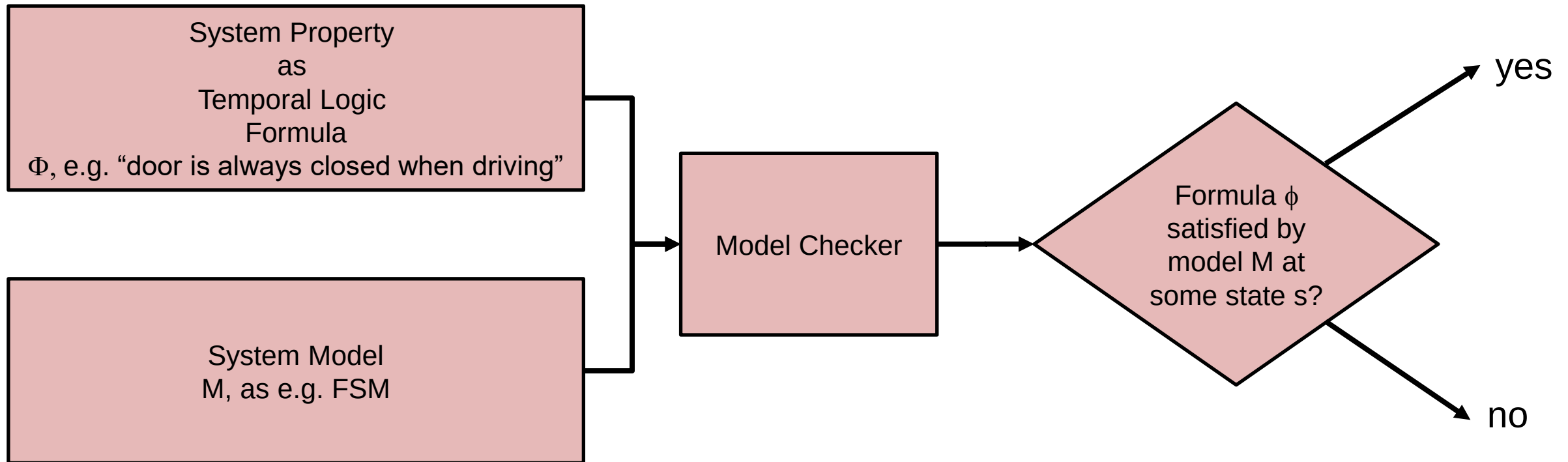
■ Generic form of a State Machine – Basically «bubbles and edges» ...

- Circles denote states
- Arrows denote transitions
- Labels on state denote properties of the state



Formal Verification using Model Checking

- **Model Checking** is an automatic verification technique for state transition systems, such as FSM, UML State Machines, Petri Nets
- In Model Checking, we can verify if assertions on a system are true or not

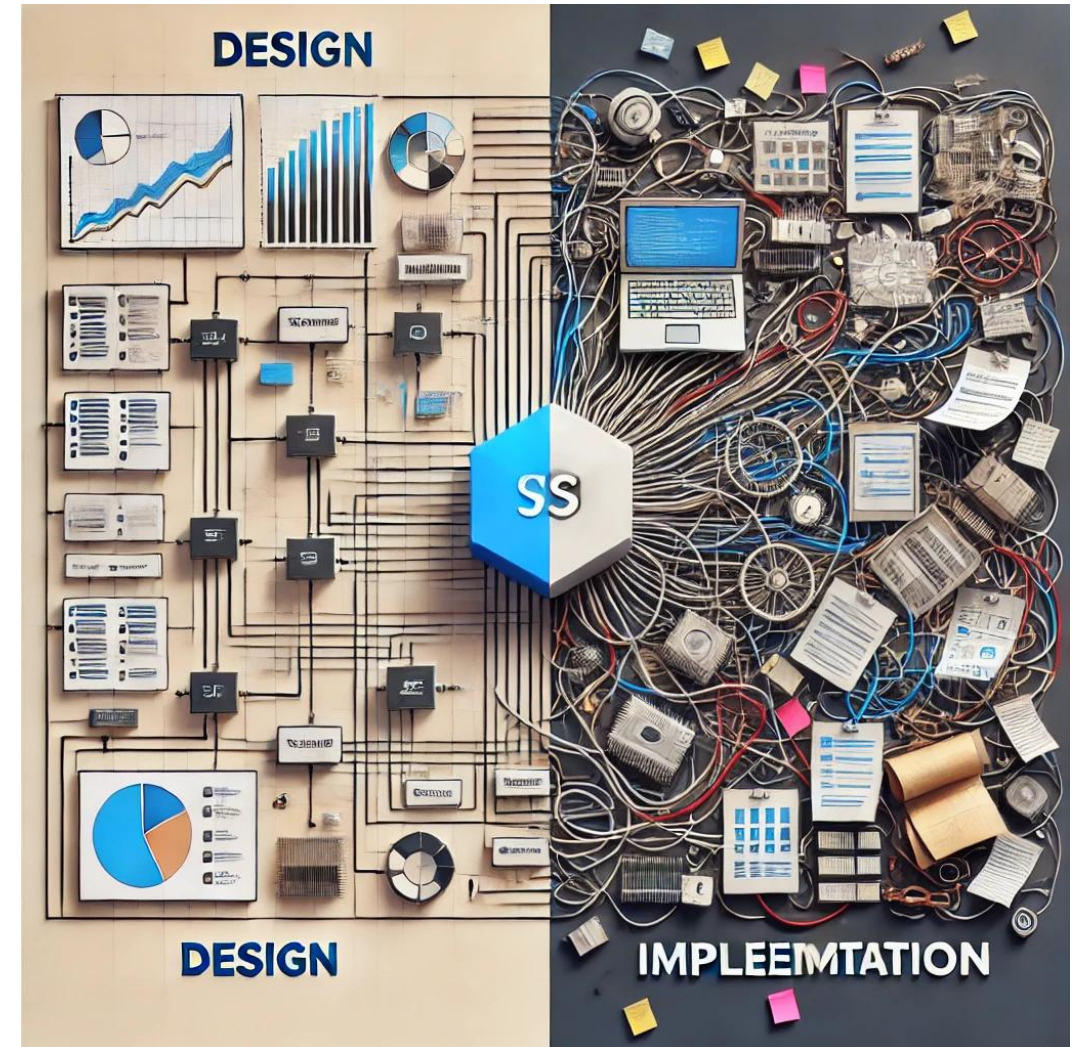


Verification with Model Checking

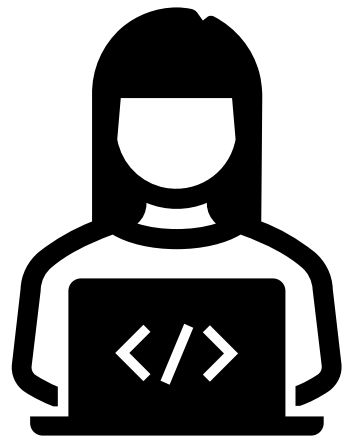
- In model checking, we use temporal logic formulas (e.g., CTL formulas)
- **Examples of temporal logic:**
 - The car cannot accelerate until the doors are closed
 - The landing gear of a plane cannot be pulled in as long as the plane is on the ground
 - After pushing an emergency stop button, the engines of a machine stop
- **A temporal logic statement is not statically true or false but can be true or false depending on the state**
- **Usually integrated into modeling tools, i.e. , a single tool can achieve the full chain of modeling, model verification, and code generation.**

Model Checking: Impact on Embedded System Development

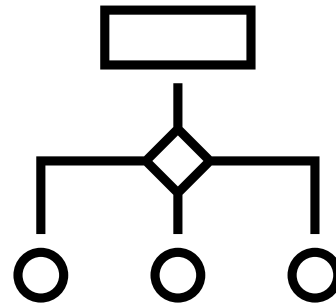
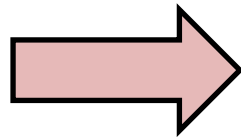
- Using Model Checking we can prove the correct functionality of our system's model
- What about the implementation? Can we ensure that the implementation follows the models?



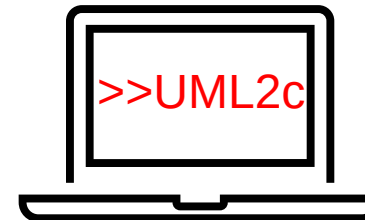
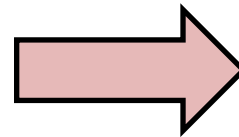
Source: ChatGPT



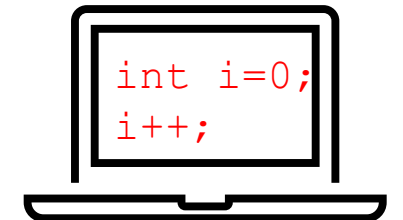
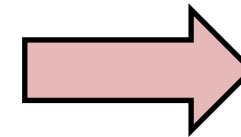
Developer



Model



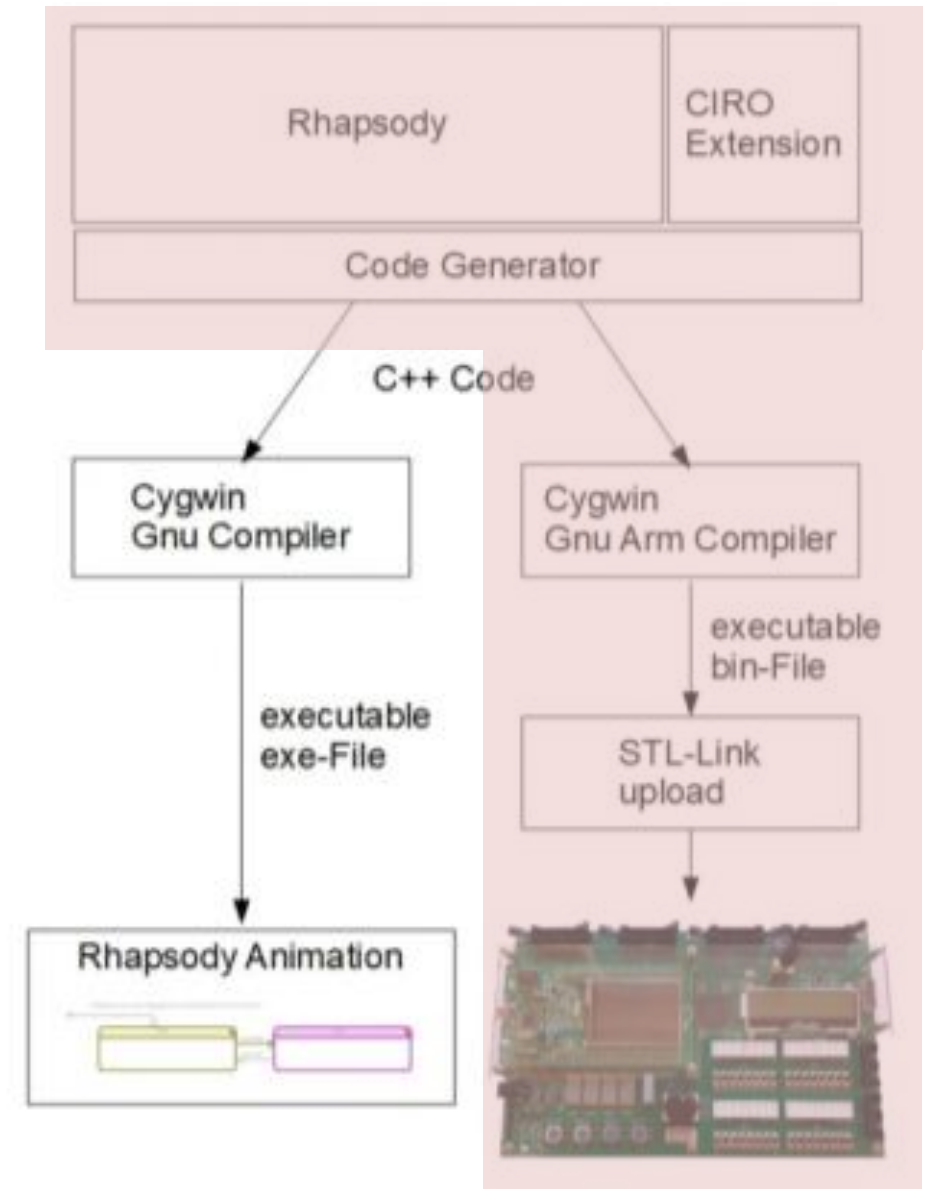
Code Generator



Generated Code

From Model to Code

- **For the demo, we will make use of the following toolchain:**
 - Rhapsody, a graphical modeling tool for development of UML models and to generate code (in our case C++)
 - CIRO extension, a Rhapsody Add-On to use UML extensions.
 - GNU compiler for ARM to compile the generated code for the target (CT board)
 - ST-Link to upload the executable file to the target



Tool Alternatives

- Found via quick web search... I didn't try them all by myself..



Innoslate
SPEC Innovations

SPEC Innovations' flagship model-based systems engineering solution can help your team reduce time-to-market, cost, and risk on even some of the most complex systems. This cloud or on-premise application uses a modern web browser, with an intuitive graphical user interface. Innoslate's full lifecycle capabilities include: • Requirements Management • Documents Management • Modelling • Discrete Event Simulator • Monte Carlo Simulator • DoDAF Models and Views • Database...

★★★★★ 3 Reviews

Compare vs. IBM Rhapsody View Software Visit Website



Enterprise Architect
Sparx Systems

Enterprise Architect is an enterprise wide solution to visualize, analyze, model, test and maintain all of your systems, software, processes, and architectures. Enterprise Architect is the ideal platform to help you to stay in control of your workspace, support your colleagues and team, enable collaboration and build confidence within your most complex projects.

★★★★★ 1 Review Starting Price: \$229

Compare vs. IBM Rhapsody View Software



Studio 5000
Rockwell Automation

The Studio 5000® environment combines elements of design into one standard framework that optimizes productivity and reduces time to commission. This intuitive integrated design environment focuses on rapid design, re-use, collaboration and virtual design. The updated user interface provides a common, modern user experience across all Studio 5000 applications. Studio 5000 Logix Designer® is an intuitive programming environment that allows users to work collaboratively to design and...

Compare vs. IBM Rhapsody View Software



Valispace
Valispace

It enables Team Collaboration on all important aspects of the Engineering Lifecycle, such as: - Requirements Management - Data-Driven Functional Design - Product Architecture - Detailed Systems Design & Simulation - Automated Verification - Live Documentation It also allows for cross-lifecycle process improvements, such as: - Agile Engineering Planning - Technical Change Management In comparison to siloed Requirement Management Systems or Model Based Systems Engineering expert...

Starting Price: €50/month/user

Compare vs. IBM Rhapsody View Software



System Composer
MathWorks

System Composer™ enables the specification and analysis of architectures for model-based systems engineering and software architecture modeling. With System Composer, you allocate requirements while refining an architecture model that can then be designed and simulated in Simulink®. Architecture models consisting of components and interfaces can be authored directly, imported from other tools, or populated from the architectural elements of Simulink designs. You can describe you...

Starting Price: \$740 per year

Compare vs. IBM Rhapsody View Software



Altair Embed
Altair

It offers thousands of pre-built models that cover a wide range of engineering disciplines, with search to identify a model that matches your design requirements, and broad target support. The Embed library of motor models and motor control algorithms, for both sensed and sensorless applications, provides the starting point to accelerate embedded motor control projects for any type of electric motor, including AC induction, BLDC, PMSM, brushed DC, or stepper motors. With Embed, users ca...

Compare vs. IBM Rhapsody View Software



MapleMBSE
Maplesoft

Systems engineering projects are much more likely to succeed if every stakeholder can easily contribute to the process, but most people don't have the training needed to use traditional SE tools, or work with the complex modeling languages found in SE practices. MapleMBSE makes broad engagement possible with Excel-based, task-specific interfaces for each activity in your SE project. By giving all stakeholders access to the process through a live, two-way connection to the model of your system...

Compare vs. IBM Rhapsody View Software



MADe
PHM Technology

Mitigate technical engineering-based risks that impact performance, operational availability and the cost of maintenance using modeling, analysis and decision support workflows. MADe is used to enable better decisions about the design and support of safety/mission-critical equipment at each stage of the product lifecycle. Reduce risk using interdependent analysis capabilities that consider the technical, operational and economic requirements of the operator and/or maintainer of the system. The...

Starting Price: \$10000.00/year

Compare vs. IBM Rhapsody View Software

And many more...

Challenges for Model-Based Code Generation

■ No manual interaction

- If generated code is edited manually and not fed back into the model, how useful the models still are?
Keeping track of the manual changes is a nightmare

■ Missing guidelines

- Unlike for handwritten code, developers tend to neglect the need for guidelines for modelling, too, leading to unstructured, not-easy-to-read models and finally, code.

■ One Solution for everything

- If the description/modeling language is kept universal, it can tempt you to want to solve every problem with it. For example, don't use state-oriented modelling for a flow-based problem.

■ Keep models simple

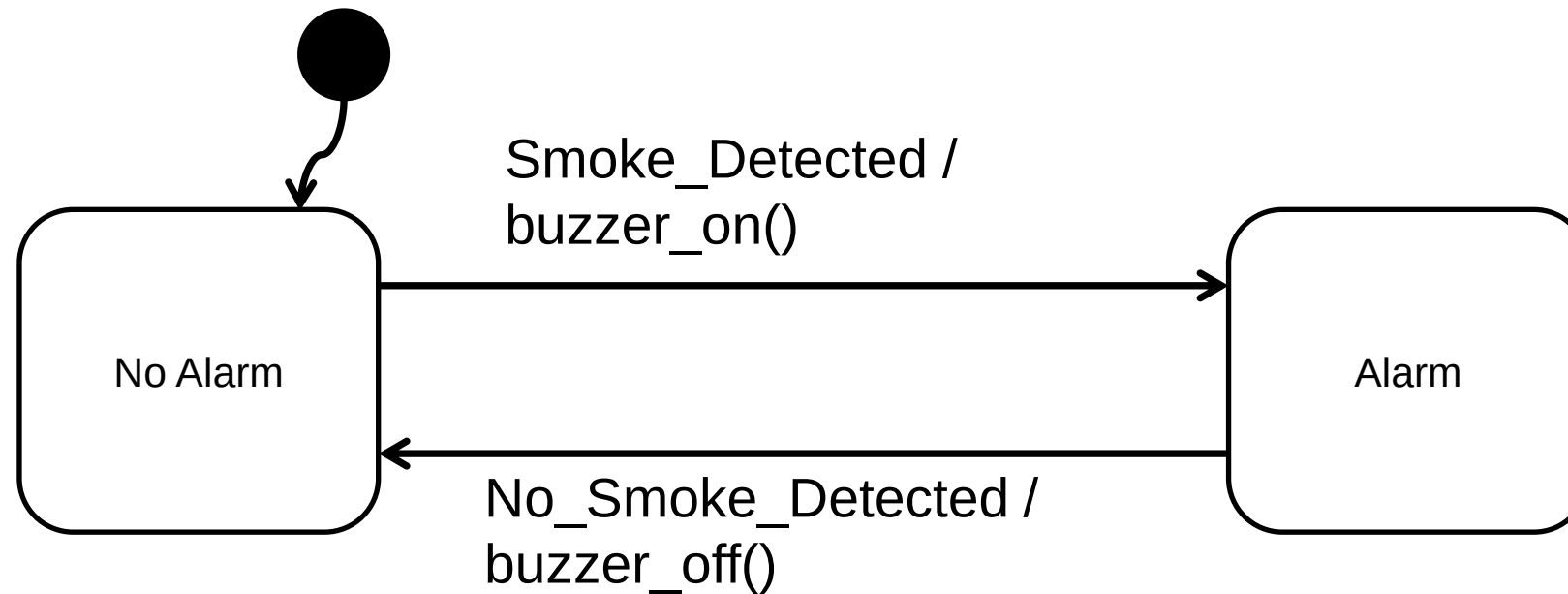
- Models which are too complex have disadvantages regarding testability, reusability and traceability.
- The generated code might suffer long compile times and will be hard to read.

■ Scripting instead of Modelling

- Some models/code generators allow scripts to be included and as a consequence, developers tend to use scripting instead of proper modelling

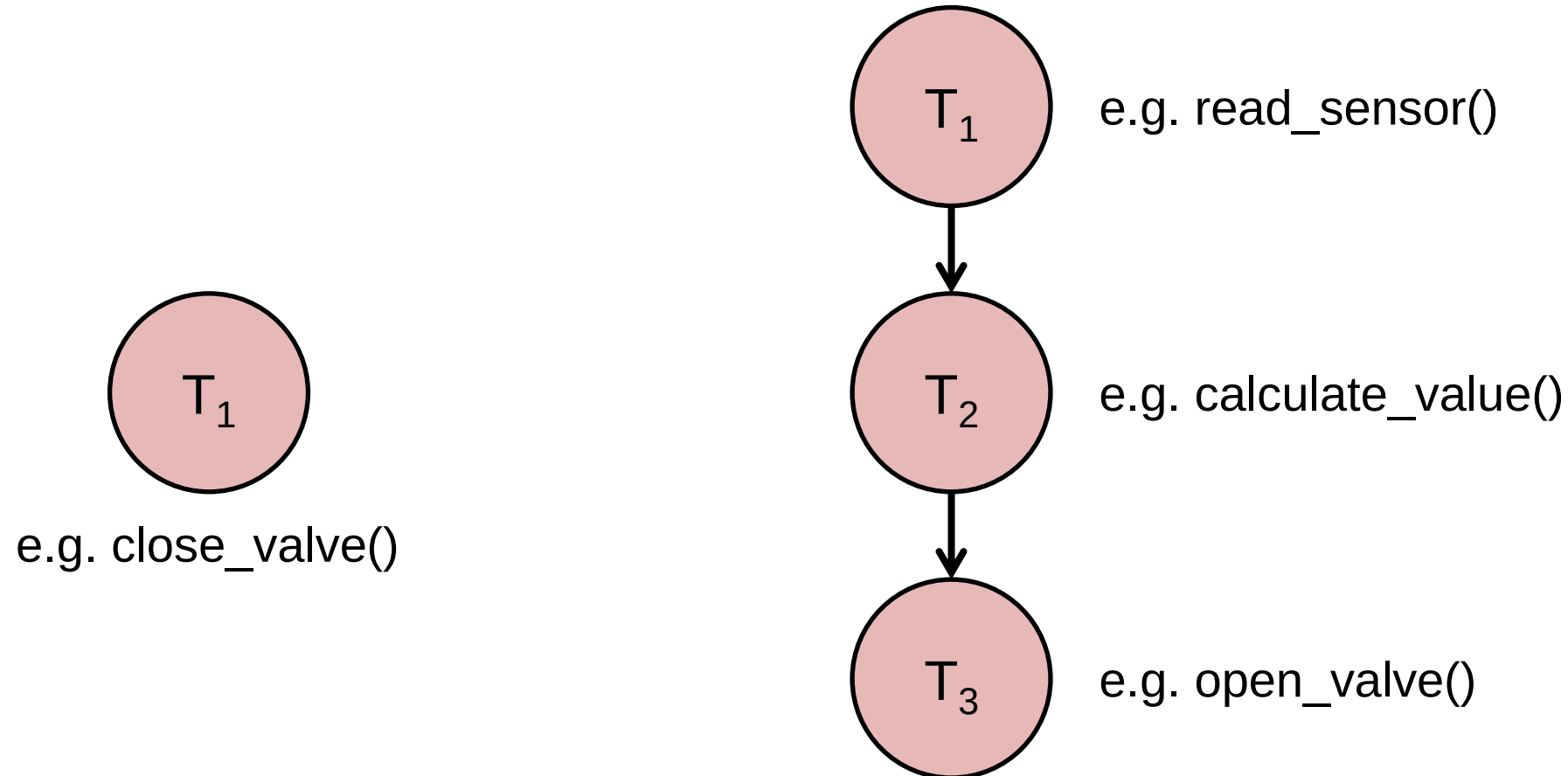
Re-Think UML State Machines

- Does the state machine represent ALL software needed for the ES?



Tasks

- A «task» is a unit of work scheduled for execution on the CPU
- Tasks may be independent or depend on each other → task graph

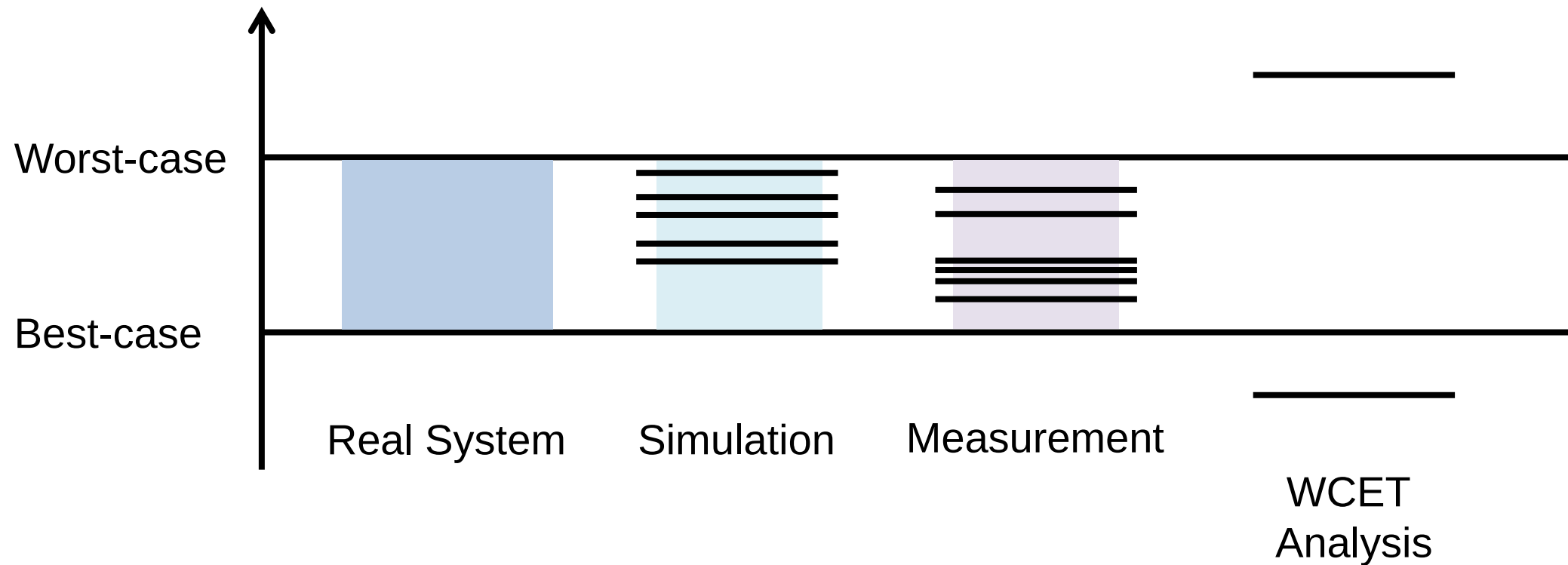


Interaction Hardware – Software

- **Is it possible to execute all tasks on the selected hardware?**
- **Also, for the worst case? How can this be determined?**

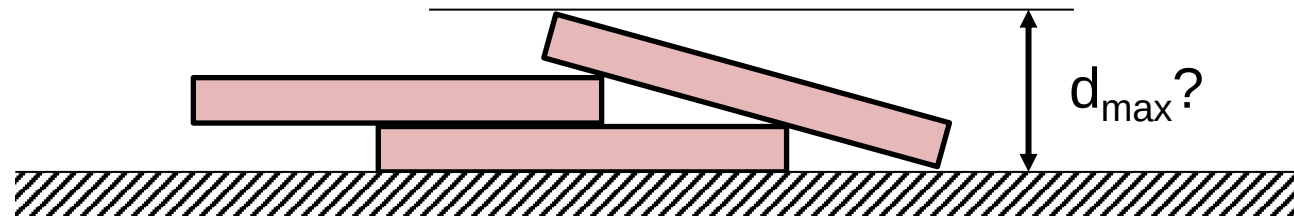
Tasks – Execution Time

- How to determine the execution time of a task?



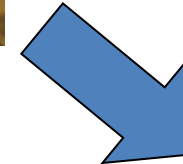
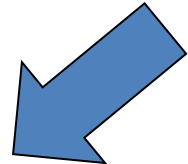
WCET Estimation is really tricky...

- Assume that you toss three coins on a table
- What is the maximum height of the stack of coins that you could observe?



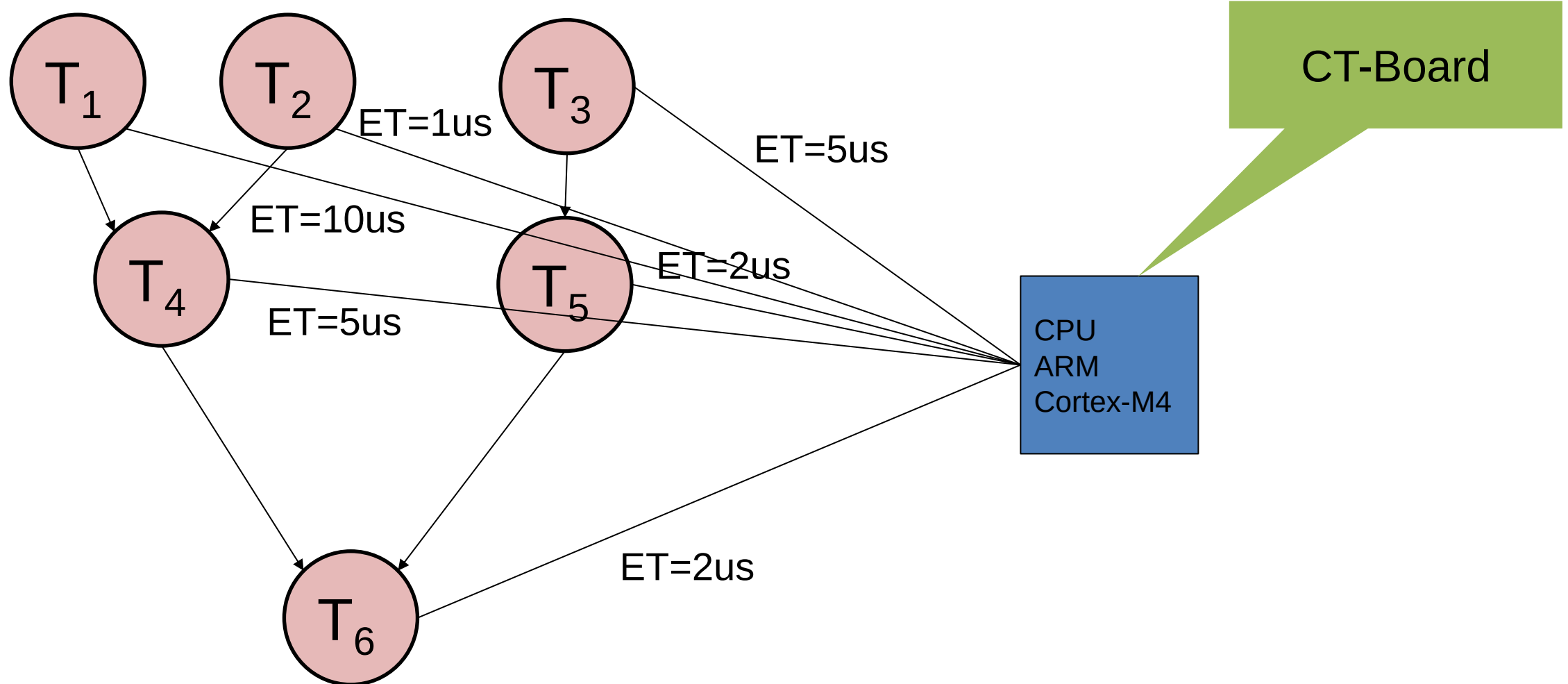
Task – Execution Time (II)

- **The execution time of a task depends on the CPU it will be executed on**
 - Performing a complicated calculation on an MCU will take much longer than on a GPU



Von © Raimond Spekking / CC BY-SA 4.0 (via Wikimedia Commons), CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=54488598>

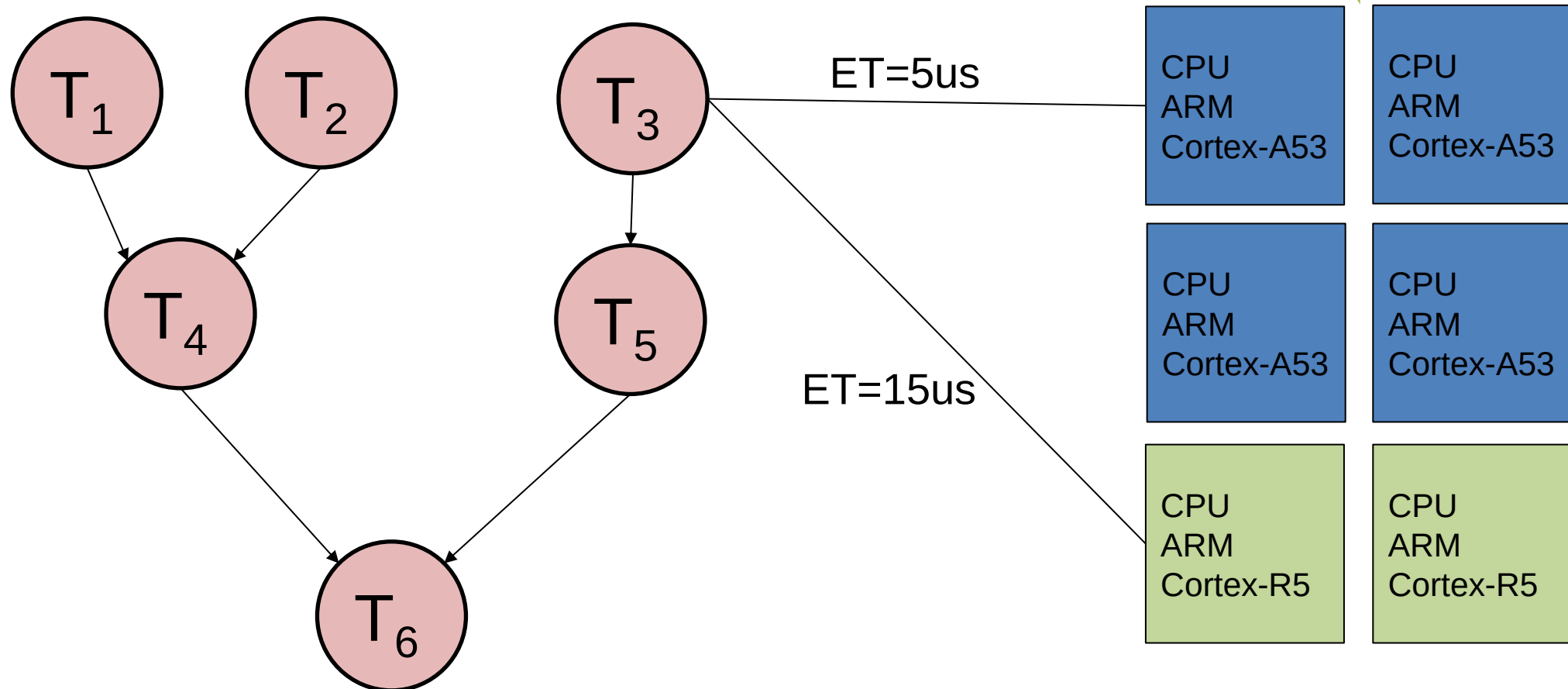
Task on a Single Processor System



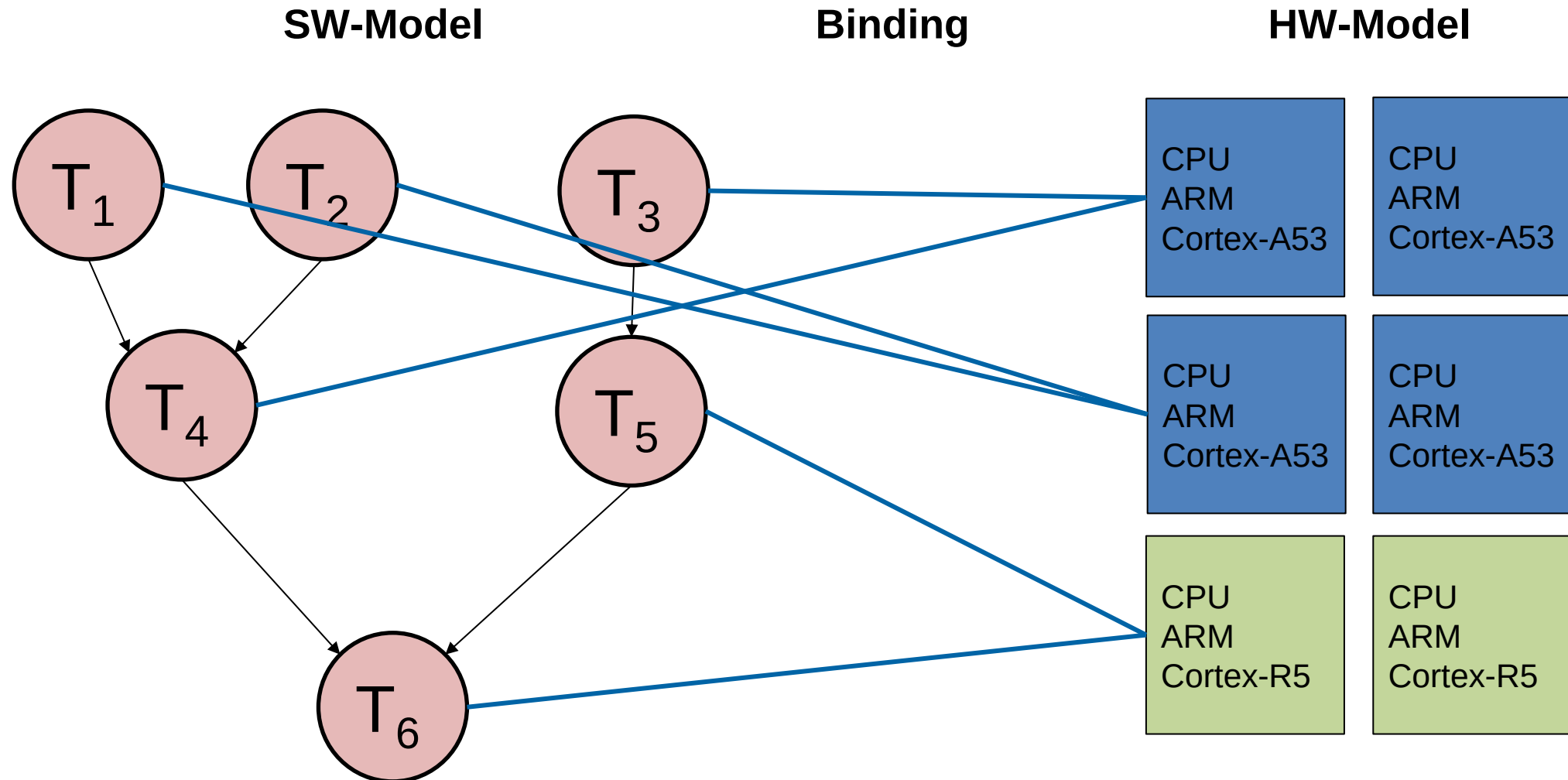
Embedded System – Multiprocessor System

ESE-Board

- Changing the processor might impact the task's execution time

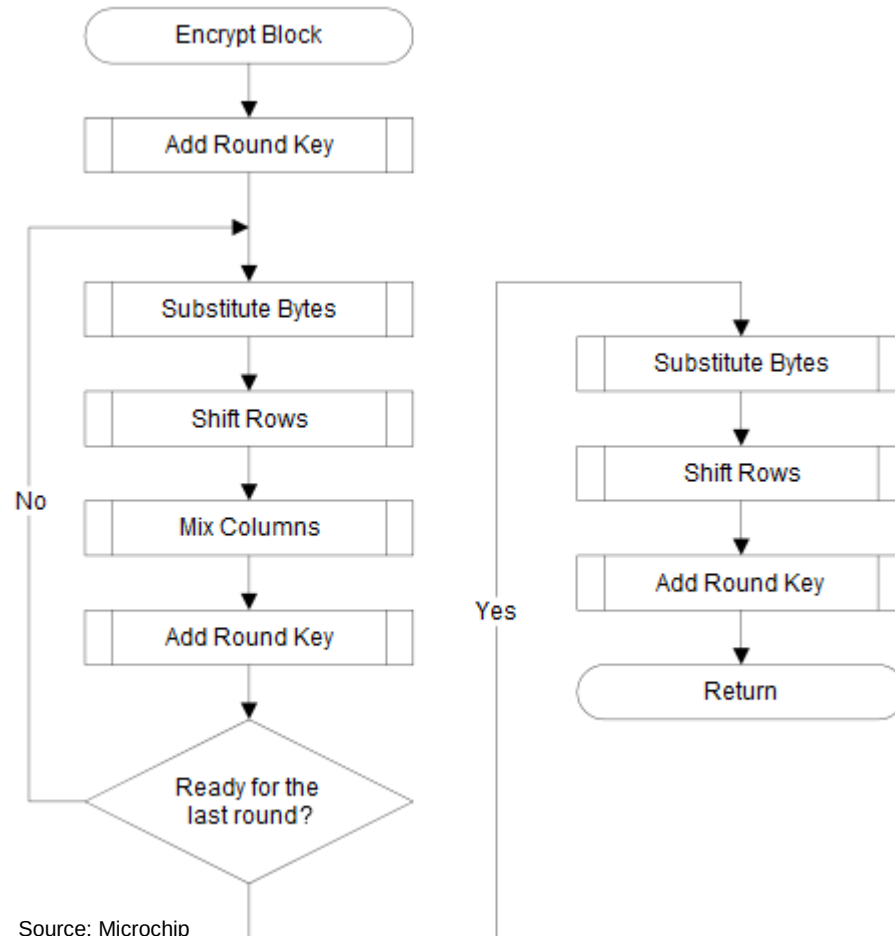


Embedded System – Multiprocessor System

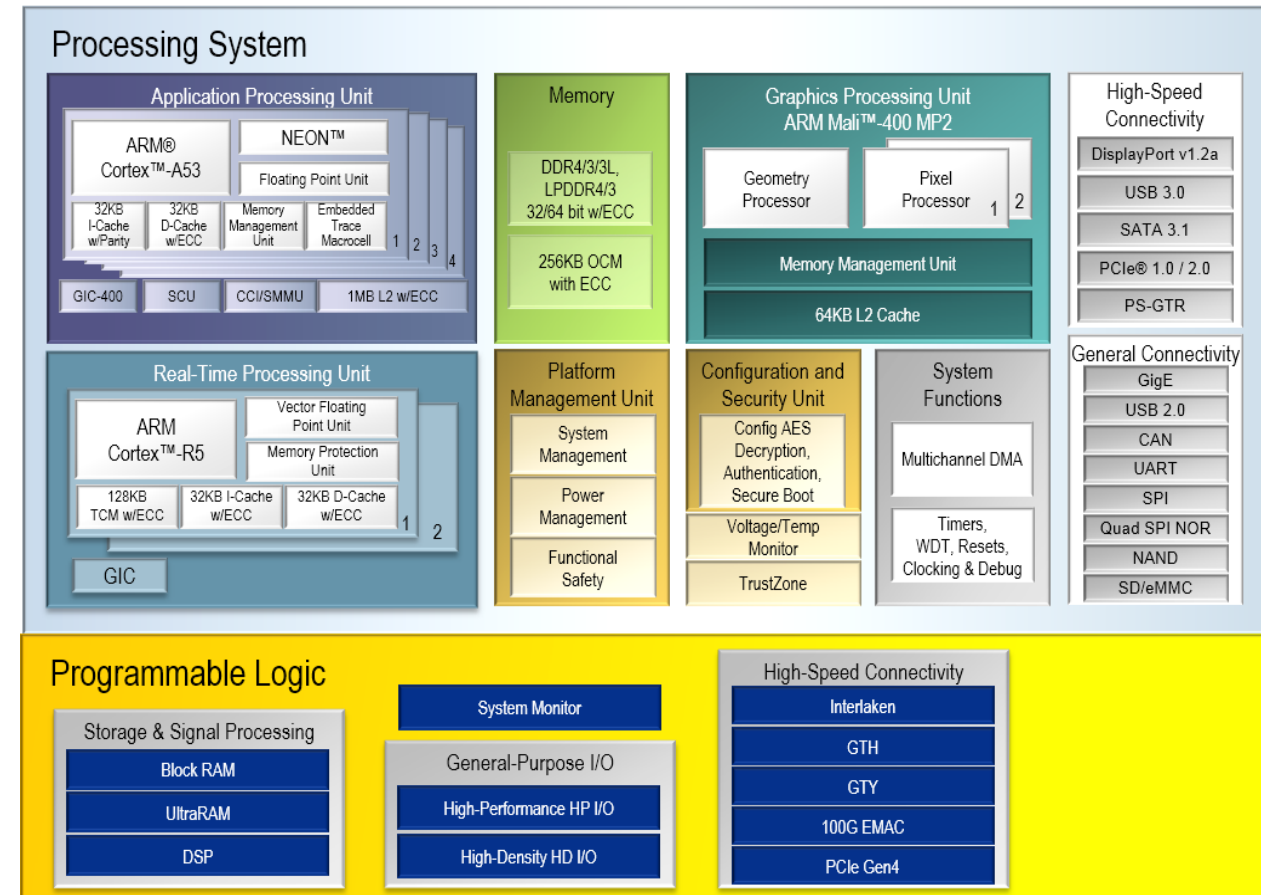


Embedded System – Multiprocessor System

SW-Model



HW-Model



Summary

- **UML State Machines can be used to model Software behavior for Embedded Systems**
- **State Machines can be formally verified, i.e. their correct functionality can be proven**
- **UML State Machines can be used for model-based source code generation for Embedded Systems**
- **Task execution times may depend on the HW resource the tasks are executed on**
- **Models for Embedded Systems include Software, Hardware, and Binding of Tasks to HW resources**